

Solution Requirements

Date	04 July 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	<i>The application must initialize a secure stateful browser session using an encrypted secret key (banking_secret_secure_key_credit_card_approval) to safely track unique user identities and session instances.</i>	High
2	Authorization levels	<i>The system must distinguish between the applicant role (authorized to clear browser context via /clear_history and submit 16 financial vectors) and the background model</i>	Medium

		<i>operator/developer (authorized to call asset loading configurations)</i>	
3	External interfaces	<i>The Flask application framework must bind to native server environments (<code>app.run</code>) and load multi-format serialization binaries (<code>model.pkl</code>, <code>scaler.pkl</code>, <code>encoder.pkl</code>) utilizing the <code>joblib</code> file-system interface</i>	High

4	Transactions processing	<i>The server pipeline must transform 16 structural form fields, execute <code>OneHotEncoder</code> structural indexing for categorical text and <code>StandardScaler</code> distribution offsets for numeric values, and push the matrix stack to the classification engine.</i>	High
5	Reporting	<i>The web layer must construct an active sliding chronological tracking table (<code>/history</code>) displaying the last 10 execution cycles including applicant names, loan parameters, confidence probabilities, and decisions</i>	Medium
6	Business rules, etc.	<i>Inputs must follow strict boundaries: Age must be between 18 and 100, finances must be non-negative, and the system must fire descriptive string reasons based on threshold calculations (e.g., Debt-to-Income weights)</i>	High

7	Compliance to laws or regulations	<i>The platform must provide explicit transparency logic mapping to fair lending acts (FCRA/ECOA) by executing deterministic explanation branches rather than acting as a non-verifiable "black box" algorithm.</i>	High
8	Other	<i>The training core must support a validation routing structure that automatically compares Logistic Regression, Decision Trees, and Random Forests against tuned XGBoost arrays to extract peak classification boundaries.</i>	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
------	--------------	-------------------------	-------------------------------------

1	Performance & Speed	<i>Processing speed of the end-to-end inference execution pipeline (data capture to calculation rendering)</i>	<i>Model prediction, scaling, and Flask route parsing must complete in < 500 milliseconds per single applicant row.</i>
2	Scalability	<i>The capacity of the backend training architecture to evaluate and optimize hyperparameter parameters across</i>	<i>The RandomizedSearch CV training pipeline must run distributed cross-validation across 24 parallel</i>

		<i>synthetic arrays.</i>	<i>job iterations via <code>n_jobs=-1</code>.</i>
3	Security & Data Privacy	<i>Data protection mechanism governing the temporary storage of applicant financial profiles in browser environments</i>	<i>Session storage and rolling user history arrays must be signed using a cryptographically hashed secret key string to prevent side-channel spoofing</i>
4	Reliability & Availability	<i>Fault-tolerant system capability to maintain platform accessibility even if underlying machine learning dependencies face disk exceptions</i>	<i>The web framework must use non-blocking exception blocks (<code>try/except</code>) on startup (<code>load_ml_assets</code>), guaranteeing 100% server uptime even if a <code>.pkl</code> asset goes missing</i>
5	Usability & Accessibility	<i>Layout standards for input forms ensuring clear data capture and graceful recovery from user mistakes</i>	<i>The system must instantly flash customized bootstrap 'danger' alerts indicating specific text errors if validation logic triggers an invalid out-of-bounds parameter</i>
6	Other	<i>Code modularity and architectural separation of predictive components from the operational application server layer</i>	<i>Training data generation and algorithm testing parameters must live completely isolated within <code>train_model.py</code> so model logic can be updated without breaking <code>app.py</code>.</i>